

Emulation of Compute Express Link (CXL) on QEMU via Ubuntu 24.04.4 LTS Report

Phase 1: Host System Preparation and Dependency Resolution

The foundation of the emulation environment is that the host must be running Ubuntu 24.04.4 LTS (Noble Numbat).

Workspace Initialization and KVM Setup

The initial step requires creating an isolated directory structure and ensuring the user has KVM hypervisor privileges. Executed from the root directory /:

```
cd /
sudo mkdir -p /opt/cxl_workspace/qemu_build
sudo mkdir -p /opt/cxl_workspace/kernel_build
sudo mkdir -p /opt/cxl_workspace/images
sudo mkdir -p /opt/cxl_workspace/tools
sudo mkdir -p /opt/cxl_workspace/qemu_install
sudo chown -R $USER:$USER /opt/cxl_workspace

# Grant the user KVM privileges for hardware acceleration
sudo usermod -aG kvm $USER
# Apply the new group membership to the current shell environment
newgrp kvm
# Verify KVM access (should output the 'kvm' group)
groups | grep kvm
```

System Update and Core Dependency Installation

Ubuntu 24.04.4 LTS relies on APT. Before installing specific dependencies, the system's package index must be updated. Executed from /opt/cxl_workspace:

```
cd /opt/cxl_workspace
sudo apt-get update
sudo apt-get dist-upgrade -y
```

The installation matrix for QEMU, the Linux kernel, and the user-space management utilities spans dozens of C/C++ libraries:

```

sudo apt-get install --no-install-recommends -y \
  build-essential git ccache flex bison bc pkg-config \
  automake autoconf libtool ninja-build meson cmake \
  python3 python3-venv python3-pip python3-dev \
  python3-sphinx python3-sphinx-rtd-theme \
  ca-certificates wget curl \
  libgl2.0-dev libpixmap-1-dev zlib1g-dev libgcrypt20-dev \
  libfdt-dev libffi-dev libslirp-dev liburing-dev libnfs-dev \
  libcurl4-gnutls-dev libzstd-dev libgudev-1.0-dev libaio-dev \
  libpmem-dev libpmem2-dev libssh-dev dbus-daemon dwarves perl \
  bridge-utils libncurses-dev libssl-dev libelf-dev libudev-dev \
  libpci-dev llvm clang asciidoc asciidoctor ruby-asciidoctor \
  xmlto libkmod-dev libsystemd-dev uuid-dev libjson-c-dev \
  libkeyutils-dev libiniparser-dev libtraceevent-dev libtracefs-dev \
  libnl-3-dev libnl-route-3-dev libibverbs-dev librdmacm-dev \
  libusb-1.0-0-dev libepoxy-dev libdrm-dev libgbm-dev libegl1-mesa-dev \
  ovmf qemu-utils libguestfs-tools \
  socat numactl iproute2 netcat-openbsd \
  sparse cscope exuberant-ctags \
  libvirglrenderer-dev libsd12-dev libgtk-3-dev \
  libvte-2.91-dev libpulse-dev libjack-dev \
  libspice-protocol-dev libspice-server-dev \
  xfslibs-dev libbpf-dev

# Pull in any remaining hidden QEMU build dependencies native to Ubuntu
sudo sed -i 's/^Types: deb$/Types: deb deb-src/' /etc/apt/sources.list.d/ubuntu.sources
sudo apt-get update
sudo apt-get build-dep qemu -y
sudo apt update

```

```

sudo apt install libguestfs-tools -y
cd ~
echo 'export LIBGUESTFS_BACKEND=direct' >> ~/.bashrc
source ~/.bashrc

```

Additional Dependencies (from the consolidated setup script)

Consolidating this process into a single script, CXL_Complete_Setup.sh, surfaced a handful of packages the original manual list did not include, needed either for guest tooling or for pieces of the build the original list happened not to exercise directly:

Package	Why it's needed
ndctl, libndctl-dev	In-guest CLI and development headers for managing CXL/NVDIMM namespaces and regions, used directly by the operations scripts
pciutils	Provides lspci, used throughout topology verification
nasm, iasl	Assembler and ACPI Source Language compiler, needed by parts of the EDK2/OVMF build toolchain
acpica-tools	ACPI table inspection/decompilation tools, useful when verifying the CEDT table QEMU/OVMF generates for CXL
cloud-image-utils	Provides cloud-localds, for building a cloud-init seed image as an alternative to virt-customize-based guest provisioning
linux-image-generic	Ensures a generic Ubuntu kernel package is present as a fallback boot option alongside the custom-built CXL kernel
nano	Convenience editor for interactive use during setup

These are additive to the original dependency list above, not replacements, every package originally listed is still required.

Phase 2: Building QEMU with Advanced CXL 3.1 and DCD Extensions

The definitive source for experimental CXL 3.1 features is the fork maintained by Jonathan Cameron.

Fetching the Emulator Source Code

```
cd /opt/cxl_workspace/qemu_build
git clone https://gitlab.com/jic23/qemu.git
cd qemu
# Checkout the latest CXL integration branch dynamically.
git checkout -b cxl-stable origin/cxl-2025-03-20
# Initialize all submodules
git submodule update --init --recursive
```

Configuring and Compiling the QEMU Binary

```
cd /opt/cxl_workspace/qemu_build/qemu
./configure \
  --target-list=x86_64-softmmu \
  --enable-debug \
  --enable-slirp \
  --enable-kvm \
  --enable-vhost-net \
  --enable-libpmem \
  --enable-virtfs \
  --enable-linux-aio \
  --enable-bpf \
  --disable-werror \
  --prefix=/opt/cxl_workspace/qemu_install

# From your qemu source directory
sed -i 's/unsigned int kind/ReplayClockKind kind/g' stubs/replay-tools.c
cd /opt/cxl_workspace/qemu_build/qemu
make -j$(nproc)
make install

# Verify the CXL-capable binary exists and is the correct version
/opt/cxl_workspace/qemu_install/bin/qemu-system-x86_64 --version
# Verify the binary exposes the required CXL devices
/opt/cxl_workspace/qemu_install/bin/qemu-system-x86_64 -device help 2>&1 | grep -i cxl
```

This configure invocation is unchanged from the original build, the consolidated setup script uses the same flag set, just without a fixed --prefix (it installs relative to its own working directory instead).

Phase 3: Building a Custom CXL-Aware Linux Kernel

For the guest OS to interpret the CXL hardware hierarchy QEMU presents, the kernel must be compiled with CXL-specific subsystem configuration. Target: Linux v6.18 LTS.

Fetching the Kernel Source

```
cd /opt/cxl_workspace/kernel_build
git clone --depth=1 --branch v6.18 https://github.com/torvalds/linux.git
cd linux
```

Kernel Configuration for Advanced CXL Emulation

Start from a KVM-guest-optimized defconfig, then inject the CXL subsystems:

```
cd /opt/cxl_workspace/kernel_build/linux
make defconfig
make kvm_guest.config
./scripts/config --enable CONFIG_EXPERT
./scripts/config --enable CONFIG_CXL_BUS
./scripts/config --enable CONFIG_CXL_PCI
./scripts/config --enable CONFIG_CXL_ACPI
./scripts/config --enable CONFIG_CXL_PMEM
./scripts/config --enable CONFIG_CXL_MEM
./scripts/config --enable CONFIG_CXL_PORT
./scripts/config --enable CONFIG_CXL_REGION
./scripts/config --enable CONFIG_CXL_MEM_RAW_COMMANDS
./scripts/config --enable CONFIG_CXL_SUSPEND
./scripts/config --enable CONFIG_ZONE_DEVICE
./scripts/config --enable CONFIG_DEV_DAX
./scripts/config --enable CONFIG_DEV_DAX_CXL
./scripts/config --enable CONFIG_DEV_DAX_PMEM
./scripts/config --enable CONFIG_FS_DAX
./scripts/config --enable CONFIG_LIBNVDIMM
./scripts/config --enable CONFIG_BLK_DEV_PMEM
./scripts/config --enable CONFIG_MEMORY_HOTPLUG
./scripts/config --enable CONFIG_MEMORY_HOTPLUG_DEFAULT_ONLINE
./scripts/config --enable CONFIG_MEMORY_HOTREMOVE
./scripts/config --enable CONFIG_NUMA
./scripts/config --enable CONFIG_ACPI_NUMA
./scripts/config --enable CONFIG_EXT4_FS
./scripts/config --enable CONFIG_DEBUG_FS
./scripts/config --enable CONFIG_CXL_REGION_INVALIDATION_TEST
./scripts/config --enable CONFIG_TRACING_SUPPORT
./scripts/config --enable CONFIG_GENERIC_TRACER
./scripts/config --enable CONFIG_FTRACE
./scripts/config --enable CONFIG_TRACE_CLOCK
./scripts/config --enable CONFIG_RING_BUFFER
./scripts/config --enable CONFIG_EVENT_TRACING
./scripts/config --enable CONFIG_DYNAMIC_DEBUG
```

Additional Kernel Options (from the consolidated setup script)

A few further options were found useful when building the same kernel for the fuller multi-device, multi-topology tracks, beyond what the original single-device demo required (RAS/error-reporting options such as CONFIG_RAS, CONFIG_PCIEAER, and CONFIG_EDAC are not repeated here, see the companion PMEM document for that discussion):

Config Option	Purpose
CONFIG_TRANSPARENT_HUGEPAGE	Improves memory-management efficiency for larger CXL-backed regions
CONFIG_DEV_DAX_KMEM	Allows a DAX device to be hot-added into the general kernel memory pool, relevant to the volatile/System-RAM track
CONFIG_MHP_DEFAULT_ONLINE_TYPE	Sets the default policy for how hot-added memory blocks are onlined
CONFIG_VIRTIO_PCI, CONFIG_VIRTIO_BLK	Explicit virtio block/Pci support for the guest disk, made explicit rather than relying on defconfig defaults
CONFIG_E1000E	Alternate NIC driver, useful when virtio-net is not the desired guest network path

Kernel Compilation

```
cd /opt/cxl_workspace/kernel_build/linux
make olddefconfig
# Ensure the destination directory exists before attempting to install modules
mkdir -p /opt/cxl_workspace/images/modules
make -j$(nproc) bzImage modules
make modules_install INSTALL_MOD_PATH=/opt/cxl_workspace/images/modules
cp arch/x86/boot/bzImage /opt/cxl_workspace/images/bzImage-6.18-cxl
```

The resulting compressed kernel executable is located at arch/x86/boot/bzImage.

Phase 4: UEFI Firmware Integration via OVMF

The firmware initializing the virtual hardware must generate the CXL Early Discovery Table (CEDT). The default QEMU firmware, SeaBIOS, cannot do this, OVMF is a strict, non-negotiable requirement.

```
cd /opt/cxl_workspace/images
# Verify paths using a proper single-line fallback
ls /usr/share/OVMF/OVMF_*.fd 2>/dev/null || find /usr/share -name 'OVMF_CODE.fd' 2>/dev/null

cp /usr/share/OVMF/OVMF_CODE*.fd .
cp /usr/share/OVMF/OVMF_VARS*.fd .

mv OVMF_CODE_4M.fd OVMF_CODE.fd 2>/dev/null || true
mv OVMF_VARS_4M.fd OVMF_VARS.fd 2>/dev/null || true

# Verify they are regular files, not symlinks
file ./OVMF_CODE.fd
file ./OVMF_VARS.fd
```

Note: the consolidated setup script offers an alternative to this apt-package approach, building OVMF directly from edk2 source, which is useful if the packaged OVMF version is too old to generate a correct CEDT for the CXL topology being emulated, but the apt-based approach above remains the simpler default and is what this environment uses.

Phase 5: Guest OS Root Filesystem Construction

Using a minimal Ubuntu Noble Numbat cloud image.

Downloading and Expanding the Virtual Disk

```
cd /opt/cxl_workspace/images
wget https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img
qemu-img convert -O qcow2 noble-server-cloudimg-amd64.img cxl-guest.qcow2
qemu-img resize -f qcow2 cxl-guest.qcow2 20G
```

Hardening the Guest Image for Direct Boot and CXL Development

Cloud images rely on cloud-init for metadata ingestion, which is bypassed here. To ensure the guest boots flawlessly and has network connectivity, a password is injected, network configuration generated,

SSH rules enforced, and custom kernel modules synchronized. LIBGUESTFS_BACKEND=direct avoids nested-KVM permission panics when virt-customize prepares its supermin appliance:

```
cd /opt/cxl_workspace/images
# 1. Create a static Netplan configuration file
cat << 'EOF' > 01-dhcp.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    all_eth:
      match:
        name: "e*"
      dhcp4: true
EOF

export LIBGUESTFS_BACKEND=direct
sudo apt-get install linux-image-generic -y

# 2. Inject root password, bypass cloud-init, and setup console
sudo virt-customize -a cxl-guest.qcow2 --root-password password:cxladmin
sudo virt-customize -a cxl-guest.qcow2 --run-command 'touch /etc/cloud/cloud-init.disabled'
sudo virt-customize -a cxl-guest.qcow2 --run-command 'systemctl enable serial-getty@ttyS0.service'

# 3. Re-enable root SSH password login (disabled by default on cloud images)
sudo virt-customize -a cxl-guest.qcow2 --run-command 'sed -i "s/^#*PermitRootLogin.*/PermitRootLogin yes/" /etc/ssh/sshd_config'
sudo virt-customize -a cxl-guest.qcow2 --run-command 'sed -i "s/^#*PasswordAuthentication.*/PasswordAuthentication yes/" /etc/ssh/sshd_config'

# 4. Inject the Netplan config to guarantee network on boot
sudo virt-customize -a cxl-guest.qcow2 --copy-in 01-dhcp.yaml:/etc/netplan/

# 5. Synchronize the custom 6.18 kernel modules into the guest's /lib directory
sudo virt-customize -a cxl-guest.qcow2 \
  --copy-in /opt/cxl_workspace/images/modules/lib/modules:/lib/
```

Alternative guest-provisioning path: the consolidated setup script can instead build a cloud-init seed image (cloud-localds, requiring the cloud-image-utils package noted above) and attach it as a second virtio-blk drive, which regenerates more cleanly on repeat runs than direct qcow2 image mutation via virt-customize. Both approaches produce an equivalent, network-reachable guest.

Phase 6: Executing QEMU CXL Emulation Topologies

-cpu host,migratable=off ensures migration boundaries do not mask hardware capability features. Because the custom CXL kernel boots directly without an initramfs, root=LABEL=cloudimg-rootfs rootwait is required so the kernel waits for the VirtIO block device to initialize before mounting, eliminating VFS kernel panics.

Before executing QEMU, the backing files are created and any stale QMP socket is removed. The volatile memory backend is not dd'd, since QEMU maps that directly to system RAM:

```
cd /opt/cxl_workspace/images
dd if=/dev/zero of=cxl-mem0.raw bs=1M count=1024
dd if=/dev/zero of=cxl-lsa0.raw bs=1M count=256
dd if=/dev/zero of=cxl-vmem0.raw bs=1M count=512

chmod 660 /opt/cxl_workspace/images/cxl-*.raw
[ -e /tmp/qmp-sock ] && rm -f /tmp/qmp-sock
cp /opt/cxl_workspace/kernel_build/linux/arch/x86/boot/bzImage \
  /opt/cxl_workspace/images/bzImage-6.18-cxl
```

Useful Extras

```
# Fix the networkd-wait-online hang permanently (run inside the VM)
systemctl disable systemd-networkd-wait-online.service
systemctl mask systemd-networkd-wait-online.service

# Install ndctl inside the guest image directly, if not already present
cd /opt/cxl_workspace/images
sudo virt-customize -a cxl-guest.qcow2 --run-command 'apt-get install -y ndctl'
```

Emulation Scenario: Standard CXL Type 3 Memory Expansion

This baseline topology models a single host bridge, a single root port, and one directly attached CXL Type 3 persistent-memory endpoint, the starting point this environment supports before scaling up to the fuller multi-switch tracks:

```
cd /opt/cxl_workspace/images
/opt/cxl_workspace/qemu_install/bin/qemu-system-x86_64 \
-machine q35,cxl=on,accel=kvm \
-cpu host,migratable=off \
-smp 4 \
-m 8G,maxmem=16G,slots=4 \
-drive if=pflash,format=raw,readonly=on,file=../OVMF_CODE.fd \
-drive if=pflash,format=raw,file=../OVMF_VARS.fd \
-kernel ./bzImage-6.18-cxl \
-append "root=/dev/vda1 rootwait rootdelay=5 console=ttyS0 rw cloud-init=disabled" \
-drive file=./cxl-guest.qcow2,format=qcow2,if=none,id=hd0 \
-device virtio-blk-pci,drive=hd0,bus=pcie.0 \
-netdev user,id=net0,hostfwd=tcp::2222-:22 \
-device virtio-net-pci,netdev=net0,bus=pcie.0 \
-nographic \
-d guest_errors \
-qmp unix:/tmp/qmp-sock,server=on,wait=off \
-object memory-backend-file,id=cxl-mem0,share=on,mem-path=./cxl-mem0.raw,size=1G \
-object memory-backend-file,id=cxl-lsa0,share=on,mem-path=./cxl-lsa0.raw,size=256M \
-object memory-backend-ram,id=vmem0,size=512M \
-device pxb-cxl,bus_nr=12,bus=pcie.0,id=cxl.1 \
-device cxl-rp,port=0,bus=cxl.1,id=root_port13,chassis=0,slot=2 \
-device cxl-type3,bus=root_port13,persistent-memdev=cxl-mem0,lsa=cxl-lsa0,id=cxlpmem0,sn=0x1 \
-M cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=4G,cxl-fmw.0.interleave-granularity=4k
```

From This Base Environment to Multiple Topologies

Every phase above, QEMU build, guest kernel, OVMF firmware, guest image, is built once and reused across every topology emulated in this project. Only the QEMU launch command and the in-guest provisioning steps change from one topology to the next:

Topology	Devices	Where It's Documented
Single-device pmem (this report)	1 root port, 1 Type 3 persistent-memory endpoint	Above, baseline validation of the environment
Dual-switch persistent memory	2 switches × 2 Type 3 pmem devices (4 total, 2 independent pools)	Companion PMEM document
Dual-switch volatile memory	2 switches × 2 Type 3 volatile devices (4 total, onlined as System RAM)	Companion VMEM document

This is the practical benefit of setting the environment up this way: the same Ubuntu host, the same QEMU binary, and the same guest kernel support every topology above, only the device count, switch structure, and backing-file set change between them.